Senior Flutter Developer

Interview Questions (Codebase-Specific)

Project	WowCare 2.0 (wowcare2-mobile) - v2.2.18+21
Repository	ChildcareWow / wowcare2-mobile
Branch	fix/Reports-Fixes (base: develop)
Date	16 September 2026
Sections	14
Total questions	177

Every question here is grounded in code, packages, configuration or tests found in this repository: Flutter with Dart ^3.9, flutter_bloc (Cubit), get_it, go_router 17, Dio, dartz, json_serializable, Isar, Firebase (Messaging, Crashlytics, Firestore), WorkManager, native geofencing channels, and 255 test files. Questions only, no answers. File paths are relative to the repository root.

Difficulty tags: **[Basic]** **[Intermediate]** **[Senior]**

Table of Contents

1. Project Architecture & Folder Structure 3
2. State Management (flutter_bloc / Cubit) 5
3. Widgets & UI 7
4. Navigation & Routing (go_router) 9
5. Networking, API Layer & Error Handling 11
6. Data Models, Serialization & Local Storage 13
7. Dependency Injection (get_it) 14
8. Widget Testing 15
9. Unit Testing & Integration Testing 17
10. Performance & Optimization 19
11. Dart Language 21
12. Platform Integration 23
13. Security, CI/CD & Release 25
14. Scenario / Code-Review Questions 27

1. Project Architecture & Folder Structure

The app splits code into top-level `lib/data/features`, `lib/domain/features` and `lib/presentation/features`, with cross-cutting code in `lib/core`, `lib/shared`, `lib/app`, `lib/bootstrap` and a legacy `lib/src`.

1. **[Basic]** Walk through the top-level layout of `lib/` (`app`, `bootstrap`, `core`, `data`, `domain`, `presentation`, `shared`, `src`). What belongs in each folder, and what should never go in `core`?
2. **[Basic]** Trace the Reports feature end to end: remote datasource, repository impl, repository interface, use case, cubit, screen. Which layer owns which responsibility? — *lib/data/features/reports, lib/domain/features/reports, lib/presentation/features/reports*
3. **[Basic]** Describe the startup sequence in `main()` and `AppBootstrap.initialize()`. Why does the order of `configureDependencies()`, Firebase init, `onBackgroundMessage` and `resolvePendingLaunchNotification()` matter? — *lib/main.dart, lib/bootstrap/app_bootstrap.dart*
4. **[Intermediate]** The layers are split horizontally (data/domain/presentation, each with feature folders) rather than feature-first with layers inside. What are the trade-offs of each approach for a 1,800-file codebase with many parallel teams?
5. **[Intermediate]** The domain repository `ReportsRepository.getRecentReports()` returns a plain `Future<List<...>>` and throws `Failure`, while the use case wraps the result in dartz `Either`. Where should the `Either` boundary sit, and what goes wrong if it is inconsistent across features? — *lib/domain/features/reports/usecases/get_recent_reports_usecase.dart*
6. **[Intermediate]** `abstract class UseCase<Type, Params>` is declared twice (in `generate_report_usecase.dart` and `download_report_usecase.dart`) and `NoParams` in a third file. How would you consolidate this, and what problem does the name `Type` cause? — *lib/domain/features/reports/usecases/*
7. **[Intermediate]** The dashboard module has no domain/data layer, and several dashboard cubits call repositories directly or import data-layer models (e.g. `my_attendance_cubit` importing attendance models). How would you judge whether that is pragmatic or a violation worth fixing? — *dashboard-baseline.md*
8. **[Intermediate]** `lib/core/utils` holds ~50 files including API constants, themes, five `ValueNotifier` event buses, JWT utils and a logger. How would you reorganise it, and how would you stop it becoming a dumping ground again? — *lib/core/utils/*
9. **[Intermediate]** Responsive logic lives in both `core/responsive/` and `shared/utils/responsive_helper.dart`, and there are paired widgets such as `rounded_button.dart` / `rounded_button_new.dart`. How do you run a migration like this without a big-bang rewrite?
10. **[Senior]** `core/notifications/notification_launch_handler.dart` imports `presentation/.../notification_route_resolver.dart` and `src/app_router.dart`. Why is a dependency from core to presentation a problem, and how would you enforce layer boundaries automatically (lint rules, package split, import_lint, melos packages)?
11. **[Senior]** The team documents "Eight Mandates" (GoRouter everywhere, cubits-only presentation, one responsive source, generated models, etc.) in `docs/mandate-rules.md` and audits violations in markdown registers. How would you turn these mandates into enforceable CI checks instead of documents? — *docs/mandate-rules.md*

12. **[Senior]** `analysis_options.yaml` has `flutter_lints` and every rule commented out. What would you enable first on a codebase this size, how would you roll it out without blocking every PR, and which bugs in this repo would lints like `use_build_context_synchronously` and `discarded_futures` have caught? *— analysis_options.yaml*
13. **[Senior]** Several files exceed 3,000 lines (`activity_daily_tab.dart` 4,796, `parent_messaging_thread.dart` 4,239, `app_router.dart` 4,022, `di.dart` 3,541). Propose a decomposition plan for one of them that you could land incrementally with tests as a safety net.
14. **[Senior]** A 3.6 MB file named `<svg width="1000" height="344725" xmlns=.xml` is committed under `lib/data/features/reports/repositories/`. How would you stop accidental artefacts like this with pre-commit hooks or CI?

2. State Management (flutter_bloc / Cubit)

The project has ~169 Cubits and no `Bloc` classes. States are mostly Equatable classes with status enums and hand-written `copyWith`, plus a few Dart 3 sealed state hierarchies.

15. **[Basic]** The project uses `Cubit` everywhere and never `Bloc`. When would you choose `Bloc` with events instead, and what would you lose by never using event transformers?

16. **[Basic]** Explain the difference between `BlocBuilder`, `BlocListener`, `BlocConsumer` and `BlocSelector`. `StaffDirectoryReportScreen` uses a `MultiBlocListener` with three `listenWhen` listeners and a `BlocSelector` on `isGenerating`. Why that combination? — *lib/presentation/features/reports/view/staff_details_report_screen.dart*

17. **[Basic]** Most states are one Equatable class with a status enum and `copyWith`, using `clearX` flags to reset nullable fields. Why is the `clearErrorMessage` flag needed, and what are the alternatives? — *lib/presentation/features/dashboard/view_model/dashboard_coordinator_state.dart*

```
DashboardState copyWith({DashboardStatus? status, String? errorMessage,
    bool clearErrorMessage = false}) {
  return DashboardState(
    status: status ?? this.status,
    errorMessage: clearErrorMessage ? null : (errorMessage ?? this.errorMessage));
}
```

18. **[Intermediate]** Compare the single-class-with-status-enum state style with the sealed style in `AttendanceDeleteState`. When does each work better, and how does exhaustive `switch` change the UI code? — *lib/presentation/features/attendance/view_model/attendance_delete_state.dart*

19. **[Intermediate]** `ParentChildCubit` is described as the single source of truth for the guardian's selected child. Its `props` compare codes only, and a `refreshNonce` field forces a re-emit. Why would you need a nonce, and what are the risks of partial `props`? — *lib/shared/common/app/cubits/parent_child_cubit.dart*

20. **[Intermediate]** The codebase has ~785 `isClosed` checks and helpers like `_safeEmit`. Why does emitting after `close()` throw, and is guarding every emit the right fix or a symptom of lifecycle problems?

21. **[Intermediate]** `ReportsCubit extends ReportsLookupCubit`, and the state is a `part of` the base file with ~24 fields. What are the pros and cons of sharing state through inheritance compared with composing two cubits? — *lib/presentation/features/reports/view_model/reports_cubit.dart*

22. **[Intermediate]** `PrivateModeToggleCubit` applies an optimistic update, rolls back on failure, checks authorization inside the cubit, and keeps a one-shot `successMessage` in state. Critique storing one-shot events in state. How else could you model side effects? — *lib/presentation/features/communication/view_model/private_mode_toggle_cubit.dart*

23. **[Intermediate]** Search debouncing is hand-rolled with a `Timer` in cubits and a `DebouncedSearchMixin` on `State`. How would the same thing look with `Bloc` plus `bloc_concurrency`/rxdart transformers, and how would you test each? — *lib/presentation/features/attendance/view_model/debounced_search_mixin.dart*

24. **[Senior]** `AdminDashboardCoordinatorCubit` subscribes to seven child cubits' streams in its constructor, calls `unawaited(initialize())` there, and closes the children without awaiting. Discuss the side effects in the constructor, ownership of the child cubits, and testability. — *lib/presentation/features/dashboard/view_model/admin_dashboard_coordinator_cubit.dart*

25. **[Senior]** `BottomNavigationCubit` listens to `RoleCubit.stream`, and `UnreadBadgeCubit` listens to a global `ValueNotifier` (`SchoolSwitchEvents.epoch`). When is cubit-to-cubit communication acceptable, and when should it move to a repository stream or the widget layer? — *lib/shared/common/app/cubits/bottom_navigation_cubit.dart*

26. **[Senior]** `GeofenceCubit` mixes in `WidgetsBindingObserver` and is ~1,200 lines. Should a cubit observe app lifecycle directly? How would you split its responsibilities (permissions, native channel, WorkManager scheduling, UI state)? — *lib/presentation/features/geofencing/view_model/geofence_cubit.dart*

27. **[Senior]** The project has five `ValueNotifier` event buses (`attendance_events`, `billing_events`, `school_switch_events`, ...) alongside the cubits. What problems do global static notifiers cause for testing and for memory, and what would you replace them with? — *lib/core/events/scheduling_events.dart, lib/core/utils/*_events.dart*

3. Widgets & UI

The UI is built from custom overlay dropdowns, a large shimmer library, `ValueNotifier`-driven report forms, sliver dashboards, chat widgets, and a WebView payment bridge.

28. **[Basic]** `StaffDirectoryReportScreen` holds form state in `ValueNotifier`s with `ValueListenableBuilder` and never calls `setState`. What are the benefits, and what must you remember in `dispose()`? — *lib/presentation/features/reports/view/staff_details_report_screen.dart*

29. **[Basic]** The codebase has ~1,970 `const SizedBox` usages. Explain what `const` constructors buy you at rebuild time and where `const` is impossible.

30. **[Basic]** `initState` in the report screens schedules `addPostFrameCallback` and then calls `context.read<ReportsCubit>().loadFormClassrooms(...)`. Why not call it directly in `initState`, and why check `mounted` inside the callback?

31. **[Intermediate]** `CustomMultiSelectDropdown` uses `OverlayEntry`, `LayerLink` and `CompositedTransformFollower`. In `didUpdateWidget` it calls `setState` and then `addPostFrameCallback(() => _overlayEntry?.markNeedsBuild())`. Why is the post-frame call needed? — *lib/shared/common/widgets/custom_multi_select_dropdown.dart*

32. **[Intermediate]** The admin dashboard uses a `CustomScrollView` with `SliverReorderableList`, `ReorderableDelayedDragStartListener(key: ValueKey(card))`, and `cacheExtent: 5000` so `GlobalKey` targets for `Scrollable.ensureVisible` stay built. Discuss the performance cost and alternatives. — *lib/presentation/features/dashboard/view/admin_dashboard.dart*

33. **[Intermediate]** When would you use `ValueKey`, `ObjectKey`, `UniqueKey` or `GlobalKey`? The repo has 75 `ValueKey`s, 20 `GlobalKey`s and no `PageStorageKey`. Where might `PageStorageKey` help, for example in tabbed messaging lists?

34. **[Intermediate]** `ResponsiveScope` is an `InheritedWidget` whose `updateShouldNotify` compares size class rather than width. Why is that a good optimisation? Which widgets would still rebuild on rotation? — *lib/core/responsive/responsive_scope.dart*

35. **[Intermediate]** `app.dart` calls `Device.setScreenSize(...)` inside `build()` to replace Sizer's `LayoutBuilder`, which caused a go_router "GlobalKey used multiple times" assertion. Why is a side effect in `build()` risky, and how would you fix the original issue properly? — *lib/app/app.dart, lib/app/app_sizer_init.dart*

36. **[Intermediate]** The app clamps text scale into steps from 0.8 to 1.2 by wrapping `MediaQuery`. What are the accessibility implications, and how would you test that layouts survive the largest scale? — *lib/app/app.dart*

37. **[Intermediate]** `ProfilePhotoActionSheet.show` grabs `Navigator.of(sheetCtx).context` before popping, opens a second sheet, and checks `confirmCtx.mounted` in `finally`. Explain the context-lifetime problem this works around. — *lib/shared/common/widgets/profile_photo_action_sheet.dart*

38. **[Intermediate]** `AutoPayTokenWebview` loads `assets/forte_auto_pay.html`, registers the JS handler `autopayTokenCallback`, injects `window.initializeAutoPay(...)` in `onLoadStop`, denies all permission requests and never forwards console logs. Walk through the security reasoning. — *lib/presentation/features/billing/widgets/auto_pay_token_webview.dart*

39. **[Senior]** `assets/languages/{en,hi,ur}.json` is bundled but never loaded, and strings live in constant classes and inline literals. How would you introduce `gen_l10n`/ARB localisation, including RTL for Urdu, into a codebase this size?

40. **[Senior]** `ThemeCubit extends Cubit<ThemeData>` has an unused `toggleTheme()`, and `MaterialApp.router` sets only `theme:`. `AppColors` has ~398 static constants. How would you design a theme system with `ThemeExtension` that supports dark mode without touching thousands of call sites? *— lib/core/theme/*

41. **[Senior]** `custom_shimmer_loader.dart` contains 47 shimmer classes in 2,752 lines. How would you design a composable skeleton system instead? *— lib/shared/common/widgets/custom_shimmer_loader.dart*

4. Navigation & Routing (go_router)

Routing uses a single `GoRouter` in `lib/src/app_router.dart` (~157 routes) with a `StatefulShellRoute.indexedStack`, a redirect chain, and custom notification and deep-link resolution.

42. **[Basic]** What is the difference between `context.go`, `context.push` and `context.pop` in go_router? Why does the router need `popAllShellBranchNavigators()` when `context.go` cannot pop imperatively pushed pages? — *lib/src/app_router.dart*

43. **[Basic]** Explain `StatefulShellRoute.indexedStack` with five branches (dashboard, classroom, activity, messaging, billing). How is each tab's navigation stack preserved, and what does `goBranch(0)` in `PopScope` do?

44. **[Basic]** Route constants live partly in `AppRoutes` inside `app_router.dart` and partly in `lib/core/router/app_routes.dart` (`ReportRoutes`, `KioskRoutes`, ...). What problems does this split cause, and how would you unify it?

45. **[Intermediate]** There is no auth redirect and no `refreshListenable`; the splash cubit decides where unauthenticated users go. What breaks with this design for deep links and for session expiry mid-use?

46. **[Intermediate]** Describe the redirect chain: kiosk guard (fails closed), dashboard persona guard (fails open when `RoleCubit` is unresolved), guardian student-scope guard, billing persona rewrite. Why fail closed in one place and open in the others? Is that safe?

47. **[Intermediate]** `_fastPage` keys the page by full location when the URI has a query string. Why does this make a deep link to the same path with a different query rebuild the screen? — *lib/src/app_router.dart*

```
final key = state.uri.hasQuery
    ? ValueKey<String>('${state.matchedLocation}?${state.uri.query}')
    : state.pageKey;
```

48. **[Intermediate]** `extra` is passed as untyped maps (`state.extra as Map<String, dynamic>?`), typed args classes (`ReportFormArgs`, `AddShiftRouteArgs`), and `is` checks on unions. What are the risks of `extra` for deep links, web and state restoration?

49. **[Intermediate]** The mandates say `..load()` in a GoRoute builder re-runs on back navigation. Explain why builders can run again, and where data loading should be triggered instead.

50. **[Intermediate]** `NotificationRouteResolver` (814 lines) switches on module strings like `'CLASSROOM'` and `'INDIVISUAL_MOVEMENTS'` and hardcodes paths such as `'/dashboard/staff'`. How would you make it type-safe and keep it in sync with the router? — *lib/presentation/features/notifications/routing/notification_route_resolver.dart*

51. **[Senior]** Cold-start launch resolution checks, in order: `FirebaseMessaging.getInitialMessage()`, a native intent bridge, local notification launch details, an iOS geofence tap stored in UserDefaults, and `AppLinks().getInitialLink()`. The result is kept in global `pendingNotificationDeepLink` variables. Critique this design and propose an alternative. — *lib/core/notifications/notification_launch_handler.dart*

52. **[Senior]** Deep links are converted into a `PushNotificationModel` by `pushNotificationFromDeepLinkUri` so links and pushes share one resolver. What are the benefits, and what security validation must happen before an external URL can drive navigation? — *lib/core/notifications/deep_link_parser.dart*

53. **[Senior]** `guardianStudentScopeDecision` returns null (allow) when the persona is unresolved or children aren't loaded. Could a guardian open another child's data through a crafted deep link during cold start? How would you close that window, and should the client enforce this at all? — *lib/src/app_router.dart*

54. **[Senior]** The kiosk cubits and `GeofenceCubit` were hoisted to the app root to fix `ProviderNotFoundException` in nested GoRouter routes. Explain why nested routes lose providers, and what the better scoping options are (ShellRoute builders, `BlocProvider.value` in builders).

5. Networking, API Layer & Error Handling

One shared Dio is configured with `CookieManager`, `AuthInterceptor`, `SessionExpiredInterceptor` (refresh + replay), and debug logging. `TokenRefreshService` handles single-flight refresh. `package:http` is also used directly in a few widgets.

55. **[Basic]** List the interceptors on the shared Dio in the order they are added, and explain why order matters for cookies, auth headers and 401 handling. — *lib/app/di/di.dart, lib/core/network/api_client.dart*

56. **[Basic]** `AuthInterceptor` sends each role-specific header three ways (lowercase, PascalCase, camelCase) and sends `'-'` for missing codes. Given that HTTP headers are case-insensitive, why might this exist, and how would you resolve it with the backend team? — *lib/core/network/auth_interceptor.dart*

57. **[Basic]** `ApiErrorHandler` maps Dio error types, `{success:false, message}` envelopes, 4xx body messages and status codes to user messages, and `Failure.message` is always sanitized. Why sanitize, and what does it strip? — *lib/core/utils/api_error_handler.dart, lib/core/errors/failures.dart*

58. **[Intermediate]** Walk through `SessionExpiredInterceptor.onError`: the `__retried` flag, refresh, the `noAutoReplay` opt-out, and why a failed replay surfaces the replay error rather than the original 401 and does not log out. — *lib/core/network/session_expired_interceptor.dart*

59. **[Intermediate]** Billing payments, activity creation and student/staff mutations set `extra['noAutoReplay']`, while attendance mutations send an `Idempotency-Key` and are replayed. Explain idempotency keys and when client-side replay is safe. — *lib/core/utils/idempotency_key.dart*

60. **[Intermediate]** `TokenRefreshService` uses a `Completer` set before any `await` as a single-flight guard. What race does it prevent if ten requests get 401 at once, and what would break if the Completer were assigned after the first `await`? — *lib/core/auth/token_refresh_service.dart*

61. **[Intermediate]** The refresh call uses `validateStatus: (_) => true` through the same shared Dio. Why, and what infinite loop could otherwise happen?

62. **[Intermediate]** `package:http` is used with bare `http.get` (no timeout, no auth, no cancellation) in four presentation widgets, while `FileDownloadService` uses a plain `Dio()` for pre-signed CloudFront URLs. Which approach is right for pre-signed URLs, and why must the auth interceptor be bypassed? — *lib/presentation/features/communication/shared/utils/attachment_open_helper.dart*

63. **[Intermediate]** Cancel tokens are only used in the activity media upload path. Where else in this app would you add cancellation (report generation, search, screen dispose), and how would you tie it to cubit `close()`?

64. **[Intermediate]** `getRecentReports()` runs `Future.wait` over four sources, captures each error separately, and only throws if all fail. Discuss partial-failure UX and how you would surface a degraded result to the UI. — *lib/data/features/reports/repositories/reports_repository_impl.dart*

65. **[Senior]** There is no generic retry or backoff interceptor, only a 3-attempt linear retry in the presigned PUT upload. Design a retry policy for this app: which methods and status codes, jitter, how it interacts with idempotency, and how it respects `CancelToken`.

66. **[Senior]** `_shouldIgnorePath` uses substring matching and ignores every `/notification/` path for logout. `AuthInterceptor` has a comment that `_skipAuthPaths` "silently no-ops on UAT" because of the gateway path prefix. How would you make path matching robust across DEV (port-per-service) and UAT/PROD (gateway)?

67. **[Senior]** `ApiConstants` reads `ENVIRONMENT` with default `'DEV'`, the Firebase factory reads the same key with default `'LOCAL'`, there is no QA branch in `ApiConstants`, and local dev points at per-developer devtunnels (`_LocalDev.sairam`). What bugs can this cause, and how would you redesign environment configuration? *— lib/core/utils/api_constants.dart, lib/core/firebase/firebase_config_factory.dart*

```
static String get baseUrl => isDev
    ? _devUrl(8082)
    : isProd
    ? ApiConstantsProd.baseUrl
    : isUat
    ? ApiConstantsUat.baseUrl
    : _dev.url(_dev.userPort);
```

6. Data Models, Serialization & Local Storage

Models mix `json_serializable` (31 files, 84 `.g.dart`) with 330+ hand-written `fromJson` factories. Storage uses Isar (messaging only), SharedPreferences through `StorageService`, and `flutter_secure_storage` for tokens and credentials.

68. **[Basic]** Mandate 8 says "generated models, never raw maps", yet most models have hand-written `fromJson`. What does `json_serializable` give you, and how do `@JsonKey(defaultValue: ...)` and a custom `fromJson` converter make parsing safer? — *lib/data/features/attendance/models/qr_validation_model.dart*

69. **[Basic]** Why does the project extend `Equatable` in ~470 classes instead of using `freezed`? Compare the two for immutability, `copyWith`, unions and codegen cost.

70. **[Basic]** What goes into SharedPreferences (`StorageServiceImpl`), what goes into `flutter_secure_storage`, and what goes into Isar in this app? Justify each choice. — *lib/core/storage/storage_service.dart*

71. **[Intermediate]** `ActivityFeedPageModel.fromJson` checks `if (data is! Map<String, dynamic>)` and uses `_asInt(...)` helpers, and token parsing accepts `accessToken ?? token ?? access_token`. Where does defensive parsing help, and where does it hide backend contract bugs? — *lib/data/features/activity/models/activity_feed_models.dart*

72. **[Intermediate]** `IsarCacheService` stores `_cacheSchemaVersion = 2` in SharedPreferences and deletes the `default*.isar` files on mismatch before `Isar.open`. Explain why, and how you would handle migrations that must not lose the offline `SyncQueueCollection`. — *lib/core/cache/isar_cache_service.dart*

73. **[Intermediate]** Isar 3.x is effectively unmaintained upstream. How would you evaluate migrating messaging storage to Drift, ObjectBox or Isar Community, and how would the `CacheService` abstraction help?

74. **[Intermediate]** `CacheService` exposes generic `put/get/getAll/getByField/delete/clear<T>`. What are the limits of a generic cache interface over a typed, query-capable database?

75. **[Intermediate]** Media is staged into `<documents>/pending_media_uploads/` with microsecond timestamp plus `Random.secure()` filenames and an extension allow-list, and a startup sweep deletes files older than 7 days. Why stage at all, and what edge cases (iOS container path changes, low disk) must you handle? — *lib/data/features/activity/datasources/media_staging_local_datasource.dart*

76. **[Intermediate]** `LostMediaLocalDataSource` wraps `ImagePicker.retrieveLostData()` on Android only and prefers `files` over `file`. Explain Android activity death during image picking and how this recovers it. — *lib/data/features/activity/datasources/lost_media_local_datasource.dart*

77. **[Senior]** `SessionLocalCacheWiper` clears messaging caches on logout/school switch but deliberately keeps `SyncQueueCollection`. What privacy and correctness issues does a surviving outbound queue create when a different user logs in on a shared kiosk or staff device? — *lib/core/cache/session_local_cache_wiper.dart*

78. **[Senior]** The Isar database is unencrypted in the documents directory and holds children's messaging data. Discuss the data-protection obligations of a childcare app and the options (iOS Data Protection classes, SQLCipher, encrypted boxes).

79. **[Senior]** `fresh_install_reset_service.dart` wipes the Keychain on first launch because iOS Keychain survives uninstall, using a SharedPreferences marker. What edge cases exist (iCloud/device backup restores, migration from v1 app), and how would you test this? — *lib/core/security/fresh_install_reset_service.dart*

7. Dependency Injection (get_it)

`lib/app/di/di.dart` is a single 3,541-line `configureDependencies()` with ~464 `registerFactory`, ~117 `registerLazySingleton`, and a few eager singletons and `registerFactoryParam` calls.

80. **[Basic]** Explain `registerFactory`, `registerLazySingleton`, `registerSingleton` and `registerFactoryParam` in get_it. Why are most cubits factories while `ApiClient` and repositories are lazy singletons?

81. **[Basic]** `configureDependencies` returns early if `StorageService` is already registered. Why might it be called twice, and is an early return the right guard? — *lib/app/di/di.dart*

82. **[Intermediate]** `SharedPreferences.getInstance()`, `IsarCacheService().init()` and the `PersistCookieJar` are awaited before registration. Compare that with `registerSingletonAsync` + `allReady()`. How does each affect startup time?

83. **[Intermediate]** `KioskAttendanceRemoteDataSource` is registered with `registerSingleton` (eager), while most things are lazy. What are the implications for startup and for test resets?

84. **[Intermediate]** The mandates forbid `serviceLocator` in views, so factory files such as `private_mode_toggle_cubit_factory.dart` and `communication_providers.dart` exist. Yet ~94 presentation files still call `serviceLocator` directly. Is get_it a service locator or DI here? How would you move toward constructor injection? — *lib/app/di/*

85. **[Senior]** `UnreadBadgeCubit`, `ParentChildCubit` and `GeofenceCubit` are lazy singletons in get_it but are handed to `BlocProvider(create: ...)` at the app root. What happens when that provider is disposed, and why is `BlocProvider.value` the right tool here? — *lib/app/app_providers.dart*

```
BlocProvider<ParentChildCubit>(
  create: (_) => serviceLocator<ParentChildCubit>(),
),
BlocProvider<UnreadBadgeCubit>(
  create: (_) => serviceLocator<UnreadBadgeCubit>(),
),
```

86. **[Senior]** `DashboardCubitFactory._resolveSafe<T>()` falls back to `ParentChildCubit() as T` when a type is not registered. What class of bugs does this hide, and what would you do instead? — *lib/presentation/features/dashboard/services/dashboard_cubit_factory.dart*

87. **[Senior]** How would you split `di.dart` into per-feature modules (or adopt `injectable`) so a feature team owns its registrations? What ordering constraints would you have to encode?

88. **[Senior]** Tests call `await serviceLocator.reset()` in setUp/tearDown, and some call `unregister<StorageService>()` without a tearDown. What leaks between tests when the global `GetIt.instance` is shared, and how would you design DI so widget tests don't touch a global container?

89. **[Senior]** How would you write a test that verifies the whole DI graph resolves (every registered type can be built) without starting Firebase or opening Isar?

8. Widget Testing

About 23 files use `testWidgets` (~71 cases). There is no mockito, mocktail or bloc_test: fakes are hand-written with `implements` and `noSuchMethod`. There are no pixel golden tests.

90. **[Basic]** What is the difference between `test()` and `testWidgets()`? `test/widget_test.dart` contains only `test()` cases for dashboard coordinator cubits. What does that name mislead a new developer into assuming? — *test/widget_test.dart*

91. **[Basic]** What does `tester.pumpWidget` do? Most tests wrap the widget in `MaterialApp(home: Scaffold(...))`. Why is a `MaterialApp` (or at least `Directionality`/`MediaQuery`) required?

92. **[Basic]** Explain `pump()`, `pump(Duration)` and `pumpAndSettle()`. The report screen tests do `tap`, then `pump()`, then `pump(const Duration(seconds: 3))`. When does `pumpAndSettle` time out, for example with shimmer or infinite animations? — *test/presentation/features/reports/view/staff_details_report_screen_test.dart*

93. **[Basic]** Compare `find.text`, `find.byType`, `find.byKey`, `find.byIcon` and `find.byWidgetPredicate`. Only `private_mode_toggle_test.dart` uses `find.byKey` (`Key('pm_switch')`). Why are keys more robust than text finders for a product whose copy changes? — *test/presentation/features/messaging/widgets/private_mode_toggle_test.dart*

94. **[Intermediate]** The report screen tests build `MultiBlocProvider` with eight fake cubits around `MaterialApp.router` with a real `GoRouter(initialLocation: '/form')` so `context.pop` works. Walk through why each wrapper is needed, and how you would extract a reusable `pumpApp` helper. — *test/presentation/features/reports/view/*

95. **[Intermediate]** `_MockRoleCubit extends Cubit<RoleState> implements RoleCubit` overrides getters. How would you mock a cubit's state stream with `mocktail` + `bloc_test`'s `MockCubit` and `whenListen`, and what does that give you over hand-written fakes?

96. **[Intermediate]** `staff_details_report_screen_test.dart` captures `lastGeneratedParams` in a fake repository and asserts on `ReportRequestPayload.fromParams(params).toJson()`. Is that a widget test or a contract test? What UI behaviours are still untested (validation, error snackbar, pop result)?

97. **[Intermediate]** `billing_transaction_report_screen_test.dart`'s test "Selecting explicitly generates only those students" only asserts the Payment Method dropdown exists (its own comment admits this). How would you finish it, using `tester.tap` on the overlay dropdown items? — *test/presentation/features/reports/view/billing_transaction_report_screen_test.dart*

98. **[Intermediate]** `student_attendance_per_student_report_screen_test.dart` calls `classroomDropdown.onChanged([...])` directly instead of tapping. What does bypassing the gesture skip, and when is it acceptable?

99. **[Intermediate]** Tests set the screen size with `tester.binding.setSurfaceSize` in some files and `tester.view.physicalSize`/`devicePixelRatio` in others, and `staff_card_widget_test.dart` loops widths [320, 375, 768, 1024] checking `tester.takeException()` for overflow. Compare the approaches and how you'd clean up after each.

100. **[Intermediate]** `chat_input_bar_test.dart` is the only file using `enterText`, and no test uses `drag` or scrolling. How would you test pagination in `activity_daily_tab.dart` (loads the next page within 400px of `maxScrollExtent`) with `tester.drag`/`scrollUntilVisible`?

101. **[Senior]** `xlsx_sheet_view_test.dart` polls in a loop with `tester.runAsync` because `compute()` isolates do not advance in fake time. Explain the fake-async zone in `testWidgets`, why real async I/O hangs, and how you would restructure `XlsxSheetView` to inject the parser for testability. — *test/presentation/features/reports/widgets/xlsx_sheet_view_test.dart*

```
for (var attempt = 0; attempt < 100; attempt++) {
  await tester.pump();
  if (find.byType(CircularProgressIndicator).evaluate().isEmpty) {
    return;
  }
  await tester.runAsync(
      () => Future<void>.delayed(const Duration(milliseconds: 50)));
}
```

102. **[Senior]** There are no `matchesGoldenFile` tests; `report_request_payload_golden_test.dart` is a JSON snapshot. Which widgets in this app are good golden candidates (shimmers, dashboard cards, message bubbles)? How would you deal with fonts (Inter, Manrope), platform differences on CI (ubuntu vs mac), and flaky goldens?

103. **[Senior]** No widget test wraps the app's `SizerInit`/`ResponsiveScope`, yet production widgets depend on them. What bugs can slip through, and how would you build a test harness that mirrors `app.dart`'s provider and MediaQuery stack?

104. **[Senior]** How would you test widgets that use `WidgetsBindingObserver` (e.g. `my_day_card.dart` with a 1-second `Timer.periodic`), including simulating `AppLifecycleState.paused/resumed` and making sure no timers are pending when the test ends? — *lib/presentation/features/dashboard/widgets/my_day_card.dart*

9. Unit Testing & Integration Testing

There are 255 `*_test.dart` files (~1,280 cases), mostly unit tests of repositories, datasources, use cases and cubits. There is no `integration_test/` package, and `ios/RunnerTests` is the empty Xcode template.

105. **[Basic]** Fakes throw from `noSuchMethod` with messages like `'${invocation.memberName} is not faked'`, while `widget_test.dart`'s `MockCubit` returns `null` from `noSuchMethod`. Why is the silent-null version dangerous? — *test/shared/common/app/cubit/parent_child_cubit_test.dart, test/widget_test.dart*

106. **[Basic]** Attendance cubit tests set up `expectLater(cubit.stream, emitsInOrder([isA<AttendanceDeleteInProgress>(), isA<AttendanceDeleteSuccess>()]))` before calling the action. Why must `expectLater` be registered first? — *test/presentation/features/attendance/view_model/staff_attendance_delete_cubit_test.dart*

107. **[Basic]** How do the datasource tests use a fake `HttpClientAdapter` (`_CapturingAdapter`) on a real `Dio`? What does that verify that a fake `Dio` (`FakeDio extends Fake implements Dio`) cannot? — *test/data/features/attendance/datasources/attendance_remote_datasource_test.dart*

108. **[Intermediate]** `session_expired_replay_idempotency_test.dart` never creates `SessionExpiredInterceptor`; it copies the interceptor's mutation by hand. What does this test actually prove? Rewrite it to use the real interceptor with an adapter that returns 401 then 200. — *test/core/network/session_expired_replay_idempotency_test.dart*

```
final replayOptions = original
  ..extra['__retried'] = true
  ..headers['Authorization'] = 'Bearer refreshed-token';
await dio.fetch<dynamic>(replayOptions);
// asserts Idempotency-Key survived
```

109. **[Intermediate]** `feature_flag_cubit_test.dart` never creates `FeatureFlagCubit`; it only tests the state, and the stub use cases are dead code. How would you spot tests like this in review, and could mutation testing help? — *test/presentation/features/messaging/cubit/feature_flag_cubit_test.dart*

110. **[Intermediate]** `parent_child_cubit_test.dart` flushes deferred work with `await Future<void>.delayed(Duration.zero)` and checks single-flight with `remote.callCount`. When is `Duration.zero` enough, and when would you use `fakeAsync` or `pumpEventQueue` instead?

111. **[Intermediate]** Debounce tests (`staff_cubit_search_debounce_test`, `student_cubit_search_debounce_test`) use real delays. Rewrite one with `fakeAsync` and `async.elapse()`. What are the pitfalls with Futures inside `fakeAsync`?

112. **[Intermediate]** `private_mode_toggle_cubit_test.dart` holds a `Completer` open to test concurrency (a second `toggle()` while one is in flight). Explain the technique and what race it proves is handled.

113. **[Intermediate]** `test/integration/private_mode_flow_test.dart` is a plain `test()` with a fake repository containing generated `throw UnimplementedError()` stubs. What does "integration test" mean in Flutter (`integration_test` package, `IntegrationTestWidgetsFlutterBinding`), and what would a real one for this flow look like? — *test/integration/private_mode_flow_test.dart*

114. **[Intermediate]** `message_entity_local_data_source_test.dart` uses an in-memory `_FakeCacheService` instead of Isar. What regressions could only be caught by running against a real Isar instance (`Isar.initializeIsarCore(download: true)`)?

115. **[Senior]** `guardianStudentScopeDecision` is extracted as a pure `@visibleForTesting` function and unit-tested, but the GoRouter redirect wiring around it is untested. How would you test the full redirect chain with a real `GoRouter` in a widget test? — *test/src/guardian_student_scope_guard_test.dart*

116. **[Senior]** Design a small integration test suite (`integration_test/`) for the most critical flows of a childcare app: login and persona routing, staff sign-in via QR, parent billing payment, and push-notification deep link. How would you run it on CI (Firebase Test Lab, emulators), and how would you stub the backend?

117. **[Senior]** CI runs `flutter test --coverage` but enforces no threshold and publishes no report, and the Firebase/TestFlight workflows ship on push to `stage` without running tests. How would you design coverage gates that don't encourage low-value tests? *— .github/workflows/ci.yml*

10. Performance & Optimization

Known hotspots: very large widget classes (a 2,307-line `build` method), ~1,014 `setState` calls, 1-second ticker widgets, `Image.network` without cache sizing, and `compute()` used for XLSX parsing and SHA-256 checksums.

118. **[Basic]** `parent_messaging_thread.dart` has a single `build()` method of about 2,300 lines. Why does a huge `build` hurt performance (not just readability), and how does extracting `const` child widgets limit rebuilds? — *lib/presentation/features/communication/presentation/messaging/view/parent_messaging_thread.dart*

119. **[Basic]** `activity_daily_tab.dart` calls `setState` 43 times. How would you find which `setState` calls rebuild too much, using the DevTools "Track widget rebuilds" and the performance overlay? — *lib/presentation/features/activities/view/activity_daily_tab.dart*

120. **[Basic]** The app uses `Image.network` and `NetworkImage` (15 places) with no `cacheWidth`/`cacheHeight` and no `cached_network_image`. What memory problems does decoding full-resolution classroom photos cause, and how would you fix it?

121. **[Intermediate]** `XlsxSheetView` parses workbooks with `compute(_parseWorkbook, bytes)`. What is copied between isolates, why must the function be top-level or static, and when would `Isolate.run` or a long-lived worker isolate be better? — *lib/presentation/features/reports/widgets/xlsx_sheet_view.dart*

122. **[Intermediate]** `ImageUploadService` hashes files in `compute` by streaming `sha256.bind(File(path).openRead())` and uploads by streaming `file.openRead()` with an explicit Content-Length, so a 200 MB video is never buffered. Why does S3 need the explicit length, and what happens to memory if you use `readAsBytes()`? — *lib/shared/services/image_upload/image_upload_service.dart*

123. **[Intermediate]** `my_day_card.dart` and `my_childs_day_card.dart` each run a 1-second `Timer.periodic` that reads cubit state and recomputes a countdown in the widget. What does this cost on a dashboard with many cards, and how would you redesign it? — *lib/presentation/features/dashboard/widgets/my_day_card.dart*

124. **[Intermediate]** `AutomaticKeepAliveClientMixin` is used in 10 tabs, including `activity_daily_tab` and `staff_messaging_tab`. What is the memory trade-off of keeping tabs alive, and how would you decide per tab?

125. **[Intermediate]** `RepaintBoundary` is used only 5 times (kiosk QR, message bubble). How do you decide where to add repaint boundaries, and how do you check that one helps (repaint rainbow, raster cache stats)?

126. **[Intermediate]** Pagination triggers when `position.pixels >= maxScrollExtent - 400`, and also when `maxScrollExtent <= 0`. What duplicate-request and infinite-loop risks exist, and how do the cubit's `hasMore`/`isLoadingMore` guards reduce them? — *lib/presentation/features/activities/view/activity_daily_tab.dart*

127. **[Senior]** `firebase_performance` is a dependency but never imported. What custom traces and HTTP metrics would you add (cold start, report generation, media upload stages), and how would you wire HTTP metrics into the Dio interceptor chain?

128. **[Senior]** `message_bubble_widget.dart` creates an `AnimationController` per heart animation in an `OverlayEntry` and disposes it only when the animation completes. How would you detect this kind of leak with DevTools memory snapshots or `leak_tracker`, and how would you fix it? — *lib/presentation/features/communication/shared/widgets/message_bubble_widget.dart*

129. **[Senior]** `admin_dashboard.dart` sets `cacheExtent: 5000` on a sliver list. Explain the effect on layout cost and memory, and how you would profile the dashboard's first frame in profile mode with `--trace-startup` and the timeline.

130. **[Senior]** The app's cold start awaits time-zone data, DI (SharedPreferences, Isar open, cookie jar), WorkManager, Firebase, and notification resolution before `runApp`. How would you measure and reduce time-to-first-frame, and what could be deferred? — *lib/bootstrap/app_bootstrap.dart*

11. Dart Language

The code targets Dart SDK ^3.9 and uses sealed classes with pattern matching, records, enhanced enums, extensions, `State` mixins, generics, `async*` streams, `Completer`s and many `late` fields.

131. **[Basic]** `late` appears ~480 times, e.g. `late final AnimationController _c = AnimationController(vsync: this)` and `late final Future<XlsxWorkbook> _workbook`. Explain lazy initialisation with `late final`, and the runtime error you get when a `late` field is read before assignment. — *lib/presentation/features/auth/view/sign_in_screen.dart*

132. **[Basic]** `NotificationCategory` is an enhanced enum with constructor fields and `static fromApi`. What can enhanced enums do that plain enums can't, and how should `fromApi` handle unknown backend values? — *lib/domain/features/notifications/entities/notification_category.dart*

133. **[Basic]** `ResponsiveContext on BuildContext` and `StringDisplayExt on String` are extension methods. What are the resolution rules and conflicts with extensions, and why can't an extension override an instance member? — *lib/core/extensions/, lib/core/responsive/*

134. **[Intermediate]** `VideoValidationResult` is a sealed class destructured with `case VideoValidationAccepted(:final sizeBytes, :final durationMs, :final mimeType):`. Explain object patterns and exhaustiveness checking with sealed types. — *lib/domain/features/activity/entities/video_validation_result.dart*

```
switch (result) {
  case VideoValidationAccepted(:final sizeBytes, :final durationMs, :final mimeType):
    // proceed with upload
  case VideoValidationRejected(:final reason):
    // show error
}
```

135. **[Intermediate]** `student_schedule_cubit.dart` returns `(int, int)? _boundsForDay(String day)` and reads `bounds.$1`. When are records a good fit, and when should you use a named class instead?

136. **[Intermediate]** `mixin DebouncedSearchMixin<T extends StatefulWidget> on State<T>` and `mixin ReportDownloadActions<T extends StatefulWidget> on State<T>`. What does the `on` clause guarantee, and what are the pitfalls when two mixins both override `dispose()` or `initState()`? — *lib/presentation/features/reports/widgets/report_download_actions.dart*

137. **[Intermediate]** `SelectionCubit<T>` copies the whole `Set<T>` with `Set<T>.from` on each toggle. Why is creating a new collection needed for Equatable/Bloc to detect a change? What happens if `T` doesn't implement `==`? — *lib/shared/common/cubits/selection_cubit.dart*

138. **[Intermediate]** `_onGenerate` in the report screen is declared `void ... async`. What is the difference between `void` and `Future<void>` for an async method, what happens to exceptions thrown in it, and which lint flags it? — *lib/presentation/features/reports/view/staff_details_report_screen.dart*

139. **[Intermediate]** `conversation_entity_repository_impl.dart` uses `Stream<int> watchUnreadCountForType(...) async*`, while `message_realtime_data_source.dart` deliberately uses a `StreamController`. When would you pick each? Explain cancellation and back-pressure with `async*`/`yield*`.

140. **[Intermediate]** `unawaited(...)` is used throughout (`main.dart`'s `_refreshAll`, the coordinator constructor). What does `unawaited` actually do, and what happens to errors from those futures? — *lib/main.dart*

141. **[Senior]** `TokenRefreshService` sets `_inFlight = Completer()` synchronously before any `await`. Explain Dart's event loop and microtask queue, and why that ordering makes single-flight correct in a single isolate. — *lib/core/auth/token_refresh_service.dart*

142. **[Senior]** Realtime data sources use `runZonedGuarded`, but `main()` does not. Explain Dart zones, how uncaught async errors reach `PlatformDispatcher.instance.onError`, and when a custom zone is still useful.
143. **[Senior]** ~13 classes use `final`/`base`/`interface` class modifiers. How do these modifiers help a large team protect domain types (e.g. `Failure`, repository interfaces) from misuse across library boundaries?

12. Platform Integration

Native integration includes geofencing through Method/EventChannels (Kotlin `GeofencePlugin`, Swift `GeofencePlugin.swift`), a notification launch bridge, a FLAG_SECURE channel, WorkManager, manual plugin registration in `MainActivity`, and a vendored `better_player` fork.

144. **[Basic]** What is the difference between a `MethodChannel` and an `EventChannel`? Geofencing uses `com.wowcare.geofencing/native` for commands such as `updateSchoolLocation` and `com.wowcare.geofencing/events` for status. Why use both? — *lib/presentation/features/geofencing/view_model/geofence_cubit.dart*

145. **[Basic]** `permission_handler` requests `Permission.location`, `Permission.notification`, and `manageExternalStorage`/`storage` for viewing a PDF. Which of these will be rejected by Play Store policy or is unnecessary on modern Android and iOS? — *lib/shared/common/view/pdf_view.dart*

146. **[Basic]** `MainActivity` extends `FlutterFragmentActivity`. Why does `local_auth` require that?

147. **[Intermediate]** Geofencing on Android uses `GeofenceForegroundService`, `GeofenceBootReceiver` and `GeofenceBroadcastReceiver`, and on iOS uses `CLLocationManager` region monitoring. What are the OS limits (region count, background location permission, battery), and what does App Review require for "Always" location? — *android/app/src/main/kotlin/com/wiseowl/wowcareapp/GeofencePlugin.kt, ios/Runner/GeofencePlugin.swift*

148. **[Intermediate]** WorkManager's `callbackDispatcher` is annotated `@pragma('vm:entry-point')` and runs `geofenceBackgroundTask` in a background isolate. What is unavailable in that isolate (get_it registrations, cubits, some plugins), and why is the pragma needed with tree shaking? — *lib/core/background/callback_dispatcher.dart*

149. **[Intermediate]** `firebaseMessagingBackgroundHandler` initialises `FlutterLocalNotificationsPlugin` and shows a local notification for data-only messages. Explain FCM notification vs data messages on Android and iOS, and when the background handler runs. — *lib/core/firebase/firebase_background_handler.dart*

150. **[Intermediate]** `native_notification_bridge.dart` reads launch data over `wowcare/notifications` because `getInitialMessage()` returns null under manual plugin registration. Why would manual registration break FirebaseMessaging's initial-message detection? — *lib/core/notifications/native_notification_bridge.dart*

151. **[Senior]** `MainActivity.configureFlutterEngine` registers plugins by reflection from a hard-coded `PLUGIN_CLASS_NAMES` list inside `catch (Throwable)`, because FFmpegKit threw `UnsatisfiedLinkError` on 16 KB page-size devices. What are the maintenance risks, and what better options exist (16 KB-aligned native libs, deferred/optional loading)? — *android/app/src/main/kotlin/com/example/wowcare/MainActivity.kt*

152. **[Senior]** Explain Android's 16 KB page-size requirement. What does `check_16kb_alignment.sh` verify with `llvm-readelf`, and how would you make it portable and run it in CI (it currently hardcodes a local NDK path)? — *check_16kb_alignment.sh*

153. **[Senior]** `third_party/better_player` is a vendored fork of better_player_plus 1.2.1 with Swift PiP fixes documented in `FORK.md`. What is your strategy for maintaining forked plugins (upstream PRs, git dependencies, patch files), and how do you track security updates? — *third_party/better_player/FORK.md*

154. **[Senior]** `pubspec.yaml` has a committed `dependency_overrides` that points `ffmpeg_kit_flutter_new_https` at a no-op stub, labelled "LOCAL-ONLY". What does that mean for CI and TestFlight builds, and how would you make simulator-only overrides impossible to ship (pubspec_overrides.yaml, CI guard)? — *pubspec.yaml, stubs/ffmpeg_kit_flutter_new_https*

155. **[Senior]** `ScreenshotProtection` uses a Kotlin channel for FLAG_SECURE with no iOS handler, and `DeviceIntegrityChecker`/`EmulatorDetector` are never called. How would you implement screenshot and screen-recording protection on iOS, and when is root/jailbreak detection worth the false positives?
— lib/core/security/

13. Security, CI/CD & Release

Environments are chosen with `--dart-define=ENVIRONMENT=...` (no product flavors). Android picks `google-services.json` per environment in Gradle, iOS copies `GoogleService-Info.plist` with a script, and GitHub Actions run CI, Firebase App Distribution and TestFlight.

156. **[Basic]** What are the differences between debug, profile and release build modes in Flutter? Which Makefile targets map to each, and why must performance be measured in profile mode? — *Makefile*

157. **[Basic]** What do `--obfuscate` and `--split-debug-info` do? Only the Makefile `prod-*-optimized` targets use them, not the CI workflows. How do you symbolicate Crashlytics stack traces from obfuscated builds?

158. **[Basic]** The environment is chosen with `--dart-define=ENVIRONMENT` instead of Android `productFlavors`/iOS schemes. Discuss the trade-offs: side-by-side installs, bundle IDs, app icons, and Firebase config selection. — *android/app/build.gradle.kts*

159. **[Intermediate]** `build.gradle.kts` decodes the base64 `dart-defines` Gradle property to find `ENVIRONMENT` and copies `firebase/<env>/google-services.json`. Yet a comment says Firebase is "intentionally pinned to UAT for ALL environments" while the code uses `dartEnvironment`. How would you resolve comment/code drift in build scripts? — *android/app/build.gradle.kts*

```
// Firebase is intentionally pinned to the UAT project (wowcare2-uat) for ALL
// build environments. ... only the google-services.json is forced to the uat folder.
val firebaseConfigEnvironment = dartEnvironment
```

160. **[Intermediate]** `firebase-stage.yml` decodes `ANDROID_KEYSTORE_BASE64` into a `.jks` file, writes `key.properties` from secrets, and distributes through Firebase App Distribution; `testflight-stage.yml` imports a .p12 into a temporary keychain and uploads with `altool`. Review both for secret hygiene and suggest improvements (Play App Signing, fastlane match, App Store Connect API key, OIDC).

161. **[Intermediate]** `google-services.json` and `GoogleService-Info.plist` for local/qa/uat/prod are committed. Are Firebase API keys secrets? What controls (API key restrictions, App Check, Firestore rules) actually protect the backend, given App Check activation is commented out? — *lib/core/security/firebase_app_check.dart*

162. **[Intermediate]** R8 `isMinifyEnabled` and `isShrinkResources` are on, with ProGuard keep rules for FlutterPlugin implementations and Pigeon `Messages$*` classes. Which kinds of runtime crashes happen when keep rules are missing (reflection-based plugin registration in `MainActivity`)? — *android/app/proguard-rules.pro*

163. **[Intermediate]** `AppLogger.d`/`.w` do nothing in release, `AppLogger.e` sends to Crashlytics, and `MediaUploadTrace` uses a "closed vocabulary" that replaces URLs and paths with `<url>`/`<path>`. Why does a childcare app need redaction, and how is it tested? — *lib/core/observability/media_upload_trace.dart*

164. **[Senior]** The remember-me feature stores the raw password in secure storage and returns it without authentication when no biometrics are enrolled, and debug logs include the username and password length. Critique this and propose a safer design (refresh tokens, `biometricOnly`, Keychain access control). — *lib/core/auth/secure_credential_store.dart*

```
AppLogger.d('[SecureCred] saved user="$username" pwLen=${password.length}');
...
if (enrolled.isEmpty) {
  AppLogger.d('... no enrolled biometrics - ungated read');
  return _readStored();
}
```

165. **[Senior]** There is no certificate pinning, and DEV uses cleartext `http://5.78.83.44` (allowed only in the debug manifest and through an iOS ATS exception). Would you add pinning to this app? Discuss rotation, CloudFront pre-signed URLs, and how to pin with Dio's `IOHttpClientAdapter`.

166. **[Senior]** `main.dart` sets `FlutterError.onError` and `PlatformDispatcher.onError` only after `AppBootstrap.initialize()`, and marks every uncaught async error `fatal: true`. Background handlers also log handled errors as fatal. What does that do to crash-free metrics, and what startup errors are lost? — *lib/main.dart*

167. **[Senior]** CI runs only on PRs to `stage`, the iOS build step is commented out, and release workflows build on push without tests or obfuscation. Design a CI/CD pipeline for this app: PR checks, merge gates, versioning (`2.2.18+21` vs `$GITHUB_RUN_NUMBER`), staged rollout, and rollback. — *.github/workflows/*

14. Scenario / Code-Review Questions

For each snippet: What does this code do? What is wrong or could be improved? How would you test it? All snippets are taken from this repository (lightly trimmed).

168. **[Senior]** Offline detection. Where is this used, and why can the `NetworkFailure` branch in `auth_repository_impl.dart` never run with connectivity_plus 7.x? — *lib/core/network/network_info.dart*

```
class NetworkInfoImpl implements NetworkInfo {
  NetworkInfoImpl(this._connectivity);
  final Connectivity _connectivity;

  @override
  Future<bool> get isConnected async {
    final result = await _connectivity.checkConnectivity();
    return result != ConnectivityResult.none;
  }
}
```

169. **[Senior]** Report form route. What happens on a cold deep link, on `context.go(ReportRoutes.studentDailyAttendance)` without `extra`, or on state restoration? Who owns the cubit's lifecycle? — *lib/src/app_router.dart*

```
GoRoute(
  path: ReportRoutes.studentDailyAttendanceSegment,
  builder: (context, state) {
    final args = state.extra! as ReportFormArgs;
    return BlocProvider<ReportsCubit>.value(
      value: args.cubit,
      child: StudentDailyAttendanceReportScreen(
        classrooms: args.classrooms ?? const <String>[],
      ),
    );
  },
),
```

170. **[Senior]** Token lifecycle observer. Review the timer lifetime, the parallel unawaited calls (can Firebase sign-in read a stale token while the refresh is in flight?), and testability. — *lib/main.dart*

```
void start() {
  Timer.periodic(_idleRefreshInterval, (_) {
    if (_lifecycleState == AppLifecycleState.resumed) {
      unawaited(_refreshAll());
    }
  });
}

Future<void> _refreshAll() async {
  unawaited(TokenRefreshService.instance.ensureFreshToken());
  unawaited(ServerTimeService.instance.sync());
  unawaited(serviceLocator<FirebaseAuthService>().signIn());
}
```

171. **[Senior]** Background push handler. What happens when `messageId` is null for several messages? Is `fatal: true` correct here? How would you unit-test this top-level function? —

lib/core/firebase/firebase_background_handler.dart

```
await plugin.show(
  id: message.messageId.hashCode,
  title: title,
  body: body,
  notificationDetails: const NotificationDetails(/* ... */),
);
// ...
} catch (e, stackTrace) {
  try {
    await ensureFirebaseInitialized();
    await FirebaseCrashlytics.instance
        .recordError(e, stackTrace, fatal: true);
  } catch (_) {}
}
```

172. **[Senior]** Attachment download inside a widget helper. Review layering, memory use for large files, missing timeout and cancellation, and the context use after `await`. —

lib/presentation/features/communication/shared/utils/attachment_open_helper.dart

```
if (!await cachedFile.exists()) {
  final response = await http.get(Uri.parse(attachment.remoteUrl));
  if (response.statusCode != 200) {
    if (context.mounted) {
      CoreUtils.showTopSnackBar(context: context,
          message: 'Failed to download the file', isError: true);
    }
    return;
  }
  final savedPath = await downloadService.saveFile(
    fileData: response.bodyBytes,
    fileName: cacheKey, /* ... */
  );
```

173. **[Senior]** Video initialisation in an activity media viewer. Find the lifecycle bugs (disposed controller, listener never removed, comparing position with an unknown duration). —

lib/presentation/features/activities/widgets/view_activity_media.dart

```
void _initializeVideo() async {
  _controller = VideoPlayerController.networkUrl(
      Uri.parse(widget.activity.videoUrl!));
  _controller!.addListener(() {
    if (_controller!.value.position >= _controller!.value.duration) {
      if (mounted) setState(() { _isPlaying = false; });
    }
  });
  try {
    await _controller!.initialize();
    _controller!.setVolume(1.0);
    // ...
```

174. **[Senior]** `setState` and `BuildContext` across async gaps. Which lint catches these? How would you move this logic into a cubit so the widget no longer needs `mounted` checks? —

lib/presentation/features/attendance/view/screens/student_attendance_absent_view.dart;

lib/presentation/features/students/view/student_profile_tab_cards/student_family_tab_card.dart

```
// student_attendance_absent_view.dart
final schoolCode = await _fetchCubit.resolveSchoolCode();
if (schoolCode.isEmpty) {
  setState(() { _isLoadingTagMap = false; }); // no mounted check
  return;
}

// student_family_tab_card.dart
final schoolCode = await _resolveSchoolCode();
if (studentCode.isEmpty || schoolCode.isEmpty) {
  CoreUtils.showTopSnackBar(context: context, message: msg, isError: true);
}
```

175. **[Senior]** Heart reaction animation in a message bubble. What leaks if the bubble is disposed mid-animation? Rewrite it safely. —

lib/presentation/features/communication/shared/widgets/message_bubble_widget.dart

```
final ctrl = AnimationController(
    vsync: this, duration: const Duration(milliseconds: 650));
ctrl.addStatusListener((status) {
  if (status == AnimationStatus.completed) {
    entry?.remove();
    entry = null;
    ctrl.dispose();
  }
});
Overlay.of(context, rootOverlay: true).insert(entry!);
ctrl.forward();
```

176. **[Senior]** Makefile "production" targets. Spot the release bug, and propose a CI safeguard that verifies the environment baked into a release artefact. — *Makefile*

```
prod-appbundle-optimized:
■$(FLUTTER) build appbundle \
■■--release \
■■--obfuscate \
■■--split-debug-info=build/app/profiles \
■■--tree-shake-icons \
■■--dart-define=ENVIRONMENT=UAT

prod-ios-optimized:
■$(FLUTTER) build ios --release --obfuscate \
■■--split-debug-info=build/app/profiles \
■■--tree-shake-icons --dart-define=ENVIRONMENT=UAT
```

177. **[Senior]** Presigned PUT upload. Review creating a new `Dio()` per upload without closing it, how the retry delay interacts with `cancelToken`, and how you would test a timeout and a mid-upload cancel. —

lib/shared/services/image_upload/image_upload_service.dart

```
Future<void> uploadFileToPresignedUrl({
  required File file, required String presignedPutUrl,
  Map<String, String> requiredHeaders = const {},
  CancelToken? cancelToken, /* ... */
}) async {
  final uploadClient = Dio();
  final contentLength = await file.length();
  final headers = <String, dynamic>{};
  // ... copy trimmed requiredHeaders
  headers.putIfAbsent('Content-Type', () => fallbackContentType);
  headers[Headers.contentLengthHeader] = contentLength;
  // PUT file.openRead() with up to 3 attempts, delay = attempt seconds
```